

oldver\visualize3.py

```
1
2 import numpy as np
3 import pandas as pd
4 import torch
5 import torch.nn.functional as F
6 import matplotlib
7 matplotlib.use("Agg")
8 import matplotlib.pyplot as plt
9 from PIL import Image
10
11 from config3 import (seed_everything, SEED, DEVICE, OUTPUT_DIR, FIGURES_DIR,
12                      CLASS_NAMES, NUM_CLASSES, IMAGE_SIZE, MODEL_SAVE_PATH)
13 from dataset3 import val_transform, MEAN, STD
14 from model3 import build_model
15
16
17 # — helpers —————
18
19 def denorm(t):
20     """Tensor (C,H,W) -> numpy (H,W,3) in [0,1] for display."""
21     img = t.cpu().numpy().transpose(1, 2, 0)
22     img = img * np.array(STD) + np.array(MEAN)
23     return np.clip(img, 0, 1)
24
25
26 def load_model():
27     ckpt = torch.load(MODEL_SAVE_PATH, map_location=DEVICE, weights_only=False)
28     model = build_model(use_se=ckpt.get("use_se", True), use_aux=False).to(DEVICE)
29     state = {k: v for k, v in ckpt["model_state"].items() if not k.startswith("aux")}
30     model.load_state_dict(state, strict=False)
31     model.eval()
32     return model
33
34
35 def pick_samples(n_per_class=3):
36     """Return n_per_class random images per class, reproducibly."""
37     rng = np.random.RandomState(SEED)
38     df = pd.read_csv(OUTPUT_DIR / "train_split.csv")
39     rows = []
40     for cls_idx in range(NUM_CLASSES):
41         subset = df[df.label == cls_idx]
42         chosen = subset.sample(n=min(n_per_class, len(subset)), random_state=rng)
43         rows.append(chosen)
44     df_sel = pd.concat(rows, ignore_index=True)
45
46     tensors, labels, names = [], [], []
47     for _, r in df_sel.iterrows():
48         img = Image.open(r.filepath).convert("RGB")
49         tensors.append(val_transform(img))
50         labels.append(int(r.label))
51         names.append(r.class_name)
52     return torch.stack(tensors).to(DEVICE), labels, names
53
54
55 # — feature map extraction —————
56
57 def extract_feature_maps(model, x):
58     """Run forward once, capture activations at three depths via hooks.
59     Returns dict[layer_name] = tensor (B, C, H, W)."""
60     targets = {"stem": model.stem, "inc3b": model.inc3b, "inc5b": model.inc5b}
61     acts = {}
62     hooks = []
63     for name, module in targets.items():
64         hooks.append(
65             module.register_forward_hook(
66                 lambda _m, _i, o, n=name: acts.__setitem__(n, o.detach())
67             )
68         )
69     with torch.no_grad():
70         model(x)
```

```

71     for h in hooks:
72         h.remove()
73     return acts
74
75
76 # — Grad-CAM (on inc5b – the deepest inception before GAP) —————
77
78 def compute_gradcam(model, x_single):
79     """Grad-CAM for a single image tensor (1,3,H,W). Returns (H,W) heatmap [0,1]."""
80     feats, grads = {}, {}
81     target = model.inc5b
82     fh = target.register_forward_hook(lambda _m, _i, o: feats.__setitem__("v", o))
83     bh = target.register_full_backward_hook(lambda _m, gi, go: grads.__setitem__("v", go[0]))
84
85     model.zero_grad()
86     out = model(x_single)
87     cls = out.argmax(1)
88     out[0, cls].backward()
89
90     w = grads["v"].mean(dim=(2, 3), keepdim=True) # channel weights
91     cam = F.relu((w * feats["v"]).sum(1, keepdim=True))
92     cam = F.interpolate(cam, (IMAGE_SIZE, IMAGE_SIZE), mode="bilinear", align_corners=False)
93     cam = cam[0, 0].detach().cpu().numpy()
94     cam = (cam - cam.min()) / (cam.max() - cam.min() + 1e-8)
95
96     fh.remove(); bh.remove()
97     return cam
98
99
100 # — saliency map —————
101
102 def compute_saliency(model, x_single):
103     """Vanilla gradient saliency for a single image. Returns (H,W) map [0,1]."""
104     inp = x_single.clone().requires_grad_(True)
105     model.zero_grad()
106     out = model(inp)
107     cls = out.argmax(1)
108     out[0, cls].backward()
109     sal = inp.grad.abs().max(dim=1)[0][0].cpu().numpy()
110     sal = (sal - sal.min()) / (sal.max() - sal.min() + 1e-8)
111     return sal
112
113
114 # — THE BIG GRID (9 rows × 6 cols) —————
115
116 def make_comparison_grid(model, x, labels, names, n_per_class=3):
117     """
118     Grid layout:
119         Col 0: Input (original RGB)
120         Col 1: Stem feature map (channel-mean)
121         Col 2: Inc3b feature map
122         Col 3: Inc5b feature map
123         Col 4: Grad-CAM overlay
124         Col 5: Saliency map
125     Row grouping: class 1 (3 rows) | class 2 (3 rows) | class 3 (3 rows)
126     """
127     col_titles = ["Input", "Stem", "Inc3b (mid)", "Inc5b (deep)", "Grad-CAM", "Saliency"]
128     n_rows = NUM_CLASSES * n_per_class
129     n_cols = len(col_titles)
130
131     # pre-compute feature maps (batched, fast)
132     acts = extract_feature_maps(model, x)
133
134     fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols * 3, n_rows * 2.8))
135
136     for row in range(n_rows):
137         cls_idx = labels[row]
138         cname = names[row]
139
140         # col 0 – input
141         axes[row, 0].imshow(denorm(x[row]))
142

```

```

143     # col 1,2,3 – feature maps (channel-mean heatmap)
144     for ci, layer_name in enumerate(["stem", "inc3b", "inc5b"]):
145         fm = acts[layer_name][row].mean(0).cpu().numpy()
146         axes[row, ci + 1].imshow(fm, cmap="viridis")
147
148     # col 4 – Grad-CAM overlay
149     cam = compute_gradcam(model, x[row:row + 1])
150     axes[row, 4].imshow(denorm(x[row]))
151     axes[row, 4].imshow(cam, cmap="jet", alpha=0.5)
152
153     # col 5 – saliency
154     sal = compute_saliency(model, x[row:row + 1])
155     axes[row, 5].imshow(sal, cmap="hot")
156
157     # row label on the left
158     axes[row, 0].set_ylabel(cname, fontsize=11, fontweight="bold", rotation=0,
159                             labelpad=60, va="center")
160
161     # column headers
162     for ci, title in enumerate(col_titles):
163         axes[0, ci].set_title(title, fontsize=12, fontweight="bold")
164
165     # class separators (horizontal lines between groups)
166     for cls_i in range(1, NUM_CLASSES):
167         row_idx = cls_i * n_per_class
168         for ci in range(n_cols):
169             for spine in axes[row_idx, ci].spines.values():
170                 spine.set_visible(True); spine.set_edgecolor("red"); spine.set_linewidth(2)
171
172     for ax_row in axes:
173         for ax in ax_row:
174             ax.set_xticks([]); ax.set_yticks([])
175
176     plt.suptitle("Class Comparison: Input → Feature Maps → Grad-CAM → Saliency",
177                 fontsize=15, fontweight="bold", y=1.01)
178     plt.tight_layout()
179     p = FIGURES_DIR / "class_comparison_grid.png"
180     plt.savefig(p, dpi=160, bbox_inches="tight")
181     plt.close()
182     print(f" ok {p.name} ({n_rows} rows x {n_cols} cols)")
183
184
185 # ——— compact Grad-CAM panel (3 classes × 3 samples) ———
186
187 def make_gradcam_panel(model, x, labels, names, n_per_class=3):
188     """3 rows (one per class), each row = 3 Grad-CAM overlays side by side."""
189     fig, axes = plt.subplots(NUM_CLASSES, n_per_class, figsize=(n_per_class * 4, NUM_CLASSES * 3.5))
190     for row in range(NUM_CLASSES):
191         for col in range(n_per_class):
192             idx = row * n_per_class + col
193             cam = compute_gradcam(model, x[idx:idx + 1])
194             axes[row, col].imshow(denorm(x[idx]))
195             axes[row, col].imshow(cam, cmap="jet", alpha=0.45)
196             axes[row, col].set_xticks([]); axes[row, col].set_yticks([])
197             if col == 0:
198                 axes[row, col].set_ylabel(CLASS_NAMES[row], fontsize=12,
199                                         fontweight="bold", rotation=0, labelpad=55, va="center")
200     plt.suptitle("Grad-CAM: Where does the network focus for each class?",
201                 fontsize=14, fontweight="bold")
202     plt.tight_layout()
203     p = FIGURES_DIR / "gradcam_class_panel.png"
204     plt.savefig(p, dpi=160, bbox_inches="tight")
205     plt.close()
206     print(f" ok {p.name}")
207
208
209 # ——— compact Saliency panel ———
210
211 def make_saliency_panel(model, x, labels, names, n_per_class=3):
212     """3 rows (one per class), each row = 3 saliency maps."""
213     fig, axes = plt.subplots(NUM_CLASSES, n_per_class * 2,
214                             figsize=(n_per_class * 6, NUM_CLASSES * 3.2))

```

```

215     for row in range(NUM_CLASSES):
216         for col in range(n_per_class):
217             idx = row * n_per_class + col
218             sal = compute_saliency(model, x[idx:idx + 1])
219             ci_img = col * 2          # input column
220             ci_sal = col * 2 + 1     # saliency column
221             axes[row, ci_img].imshow(denorm(x[idx]))
222             axes[row, ci_img].set_xticks([]); axes[row, ci_img].set_yticks([])
223             axes[row, ci_sal].imshow(sal, cmap="hot")
224             axes[row, ci_sal].set_xticks([]); axes[row, ci_sal].set_yticks([])
225             if col == 0:
226                 axes[row, ci_img].set_ylabel(CLASS_NAMES[row], fontsize=12,
227                 fontweight="bold", rotation=0, labelpad=55, va="center")
228
229     plt.suptitle("Saliency: Which pixels matter most for each class prediction?",
230     fontsize=14, fontweight="bold")
231
232     plt.tight_layout()
233     p = FIGURES_DIR / "saliency_class_panel.png"
234     plt.savefig(p, dpi=160, bbox_inches="tight")
235     plt.close()
236     print(f"    ok {p.name}")
237
238 # --- main ---
239
240 def main():
241     seed_everything(SEED)
242     print("=" * 70)
243     print("    VISUALIZATION - 3 classes x 3 samples comparison grid")
244     print("=" * 70)
245
246     model = load_model()
247     N_PER_CLASS = 3
248     x, labels, names = pick_samples(n_per_class=N_PER_CLASS)
249     print(f"    Loaded {len(x)} images ({N_PER_CLASS} per class)")
250
251     print("\n [1/3] Building full comparison grid (9 rows x 6 cols)...")
252     make_comparison_grid(model, x, labels, names, N_PER_CLASS)
253
254     print("\n [2/3] Grad-CAM class panel...")
255     make_gradcam_panel(model, x, labels, names, N_PER_CLASS)
256
257     print("\n [3/3] Saliency class panel...")
258     make_saliency_panel(model, x, labels, names, N_PER_CLASS)
259
260     print(f"\n    All saved to {FIGURES_DIR}")
261     print("    Done.\n")
262
263 if __name__ == "__main__":
264     main()

```